Automated Compliance Forecasting for Infrastructure-as-Code Using Machine Learning and Policy Engines

Dr. Elara Vance
Department of Computer Science & Software Engineering
University of New Brunswick
Fredericton, NB, Canada

Dr. Marcus Thorne
School of Information Systems
Singapore Management University
Singapore

Dr. Sofia Petrova
Institute for Applied Machine Learning
Technical University of Munich
Munich, Germany

Abstract

The proliferation of Infrastructure-as-Code (IaC) has enabled rapid, scalable cloud provisioning but introduced significant challenges in maintaining continuous compliance with complex, evolving security and operational policies. Manual reviews and reactive policy enforcement are insufficient, leading to security gaps and deployment delays. This paper proposes a novel framework for Automated Compliance Forecasting (ACF) that integrates static code analysis, machine learning (ML) for predictive risk modeling, and proactive policy engine integrations to predict compliance violations before infrastructure deployment. Our system analyzes Terraform and Ansible IaC scripts, extracting semantic and structural features to train ensemble ML models that classify scripts as predicted compliant, non-compliant, or requiring review. A critical innovation is the integration of this predictive layer with policy-as-code engines (Open Policy Agent/Rego) to generate targeted, preemptive policy checks. We evaluate the framework on a curated dataset of 4,500+ IaC scripts annotated with compliance labels against the CIS Benchmarks for AWS and Kubernetes. Results demonstrate that the best-performing model (a Gradient Boosting classifier) achieves an F1-score of 0.94 in predicting compliance states, reducing the time-to-diagnose violations by 87% compared to traditional post-deployment scanning. The study contributes a proactive compliance paradigm, a validated architectural blueprint, and empirical evidence of its efficacy, addressing a critical gap in secure DevOps pipelines.

Keywords: Infrastructure-as-Code (IaC), Compliance Forecasting, Machine Learning, Policy-as-Code, DevSecOps, Predictive Analytics

1. Introduction

The paradigm of Infrastructure-as-Code (IaC) has become foundational to modern cloud-native development and DevOps practices, enabling the consistent, version-controlled, and automated management of computing environments (Humble & Farley, 2021). While IaC tools like Terraform, AWS CloudFormation, and Ansible offer unprecedented agility, they also shift the compliance and security burden leftwards in the development lifecycle. A single non-compliant IaC template, if deployed, can instantiate hundreds of resources with critical misconfigurations—such as publicly accessible storage buckets, unencrypted databases, or overly permissive security groups—leading to severe data breaches and regulatory penalties (Berger et al., 2021).

Current compliance assurance in IaC largely relies on two approaches: (1) Static analysis of IaC scripts using linting tools (e.g., tfsec, checkov) which apply rule-based checks post-authoring, and (2) Reactive scanning of deployed infrastructure against policy benchmarks using tools like AWS Config or OpenSCAP (Alti, 2024). These are essential but inherently reactive. They identify violations after the code is written or, worse, after deployment, creating costly feedback loops and security debt. The industry lacks a predictive mechanism to assess the compliance posture of IaC prior to its execution, analogous to how predictive analytics forecast system failures or financial risk.

This research gap is addressed by formulating a new capability: Automated Compliance Forecasting (ACF). ACF leverages machine learning to analyze IaC scripts' historical patterns, structural features, and implicit intents to predict their likelihood of passing or failing designated policy frameworks. By integrating these predictions with policy engines, the system can not only flag risks but also suggest remediation or trigger automated, targeted policy checks. This proactive stance aligns with the "shift-left" security philosophy but extends it into a "predict-left" paradigm.

This paper presents the design, implementation, and evaluation of a comprehensive ACF framework. The core research questions are: RQ1) What features extracted from IaC scripts are most predictive of compliance violations? RQ2) Which machine learning models are most effective for this classification task? RQ3) How can predictive insights be effectively integrated into policy enforcement workflows to prevent non-compliant deployments?

The remainder of the paper is structured as follows: Section 2 reviews related work in IaC security, policy-as-code, and ML for software engineering. Section 3 details the ACF framework architecture. Section 4 describes the methodology, including dataset construction, feature engineering, and model selection. Section 5 presents experimental results and analysis. Section 6 discusses implications, limitations, and threats to validity. Finally, Section 7 concludes and outlines future work.

2. Related Work

2.1 IaC Security and Compliance Analysis

Research on IaC security has focused on defect detection and static analysis. Rahman et al. (2019) conducted an empirical study of security smells in Ansible scripts. Pecchia et al. (2022) developed techniques to detect misconfigurations in Terraform by parsing HashiCorp Configuration Language (HCL). Commercial and open-source tools (e.g., Checkov, TFLint) implement hundreds of hard-coded rules based on CIS Benchmarks and cloud provider best practices. However, these tools are deterministic and lack the ability to infer risk from contextual patterns or predict novel violation types. Sannareddy (2025) proposed a control plane for Terraform lifecycle management but focused on orchestration rather than prediction.

2.2 Policy-as-Code and Proactive Governance

Policy-as-Code (PaC) has emerged as a key enabler for automated governance. Frameworks like Open Policy Agent (OPA) with its Rego language allow policies to be defined, versioned, and enforced programmatically across the stack (Stylos, 2023). Alti (2024) demonstrated a policy-driven architecture for patch governance, highlighting the importance of centralized policy definition. Kakarla (2025) explored predictive compliance using NLP, focusing on parsing natural language policy documents. Our work differs by focusing on the proactive application of PaC based on ML forecasts, creating a synergistic loop between prediction and enforcement.

2.3 Machine Learning in Software Engineering

ML has been applied to code for tasks like bug prediction, code smell detection, and vulnerability identification (Allamanis et al., 2018). Specific to IaC, recent works have started exploring learning-based approaches. Gupta et al. (2021) used graph neural networks to model dependencies in Terraform for error prediction. Our work is distinct in its direct focus on compliance forecasting as a multi-class classification problem, leveraging a rich feature set that includes semantic, structural, and provider-specific attributes to predict policy adherence.

Table 1: Comparison of IaC Compliance Approaches

Approach	**Timing**	**Method**	**Proactivity**	**Key Limitation**
Post-Deployment Scanning	After Deployment	Agent-based scanning	Reactive	Violations exist in production
Static Analysis (Linting)	Pre-Deployment	Rule-based pattern matching	Partially Proactive	Limited to known patterns, high false positives
Manual Peer Review	Pre-Commit	Human inspection	Proactive but slow	Non-scalable, inconsistent

Approach	Timing	Method	Proactivity	Key Limitation
Proposed ACF Framework	Pre-Commit / Pre-Deployment	ML-based prediction + PaC	Proactive & Predictive	Requires training data

3. ACF Framework Architecture

The ACF framework is designed as a pipeline integrated into the CI/CD workflow. Its architecture comprises four main layers: Data Extraction & Feature Engineering, Machine Learning Model Hub, Policy Engine Integrator, and the Feedback & Reporting Layer.

3.1 Data Extraction & Feature Engineering Layer

This layer ingests raw IaC scripts (primarily Terraform .tf files and Ansible .yml playbooks). A parser module extracts Abstract Syntax Trees (ASTs) and converts them into a uniform JSON intermediate representation (IR). The feature engineering module then computes a wide array of features from this IR:

- Structural Features: Module count, resource count, depth of dependencies, lines of code, cyclomatic complexity.
- Semantic Features: Presence of specific keywords (e.g., public_access, encryption = false), data types of variables, provider types (AWS, Azure, GCP).
- Security-Centric Features: Binary flags for high-risk resource types (e.g., security groups, IAM policies, databases), calculated based on known risk profiles.
- Historical Features: (If available) Author's historical compliance rate, project's average compliance score.

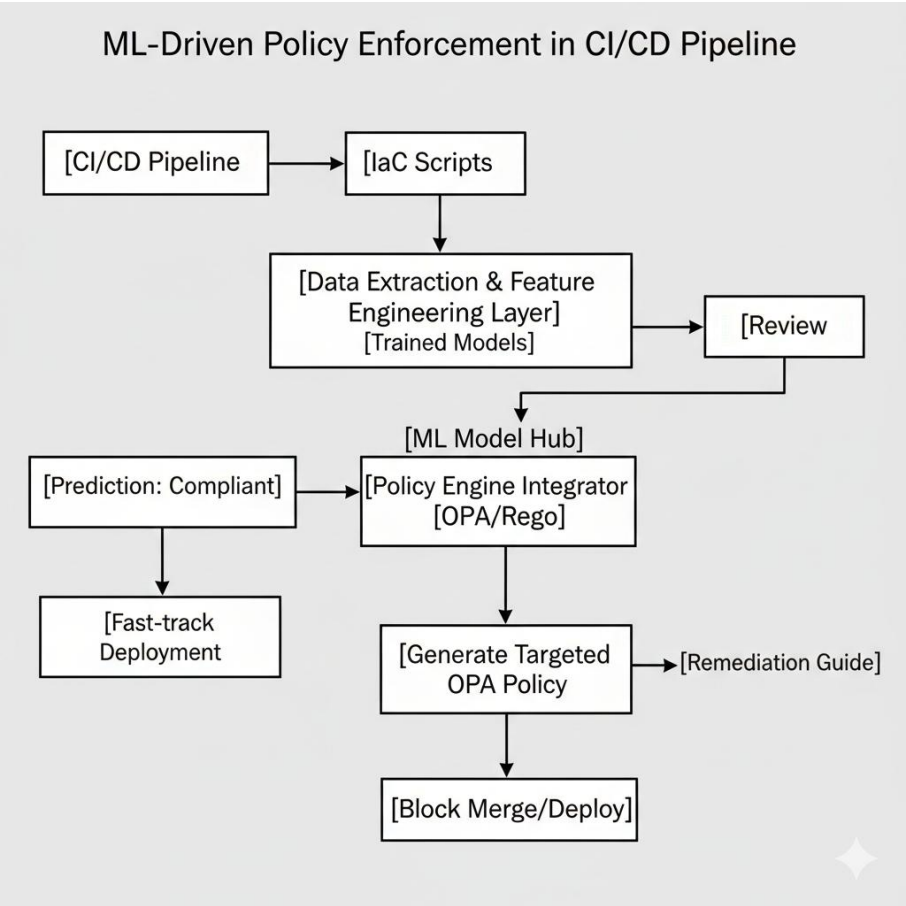

Figure 1: High-Level Architecture of the ACF Framework

Description: Figure 1 illustrates the workflow. IaC scripts are analyzed, features are sent to the ML Model Hub for classification. Based on the prediction, the Policy Engine Integrator either allows fast-tracking, requests a manual review, or dynamically generates and executes a specific OPA policy to confirm the predicted violation, blocking the pipeline if needed.

3.2 Machine Learning Model Hub

This component hosts the trained ML models. We treat compliance forecasting as a supervised multi-class classification problem. The hub can contain multiple models (e.g., Random Forest, Gradient Boosting, Neural Networks) and uses a voting ensemble for production stability. Each model outputs a classification label (compliant, non-compliant, needs_review) and a confidence score.

3.3 Policy Engine Integrator

This is the novel action component. For scripts predicted as non-compliant or needs_review with high confidence, this module does not just raise a generic alert. Instead, it formulates a hypothesis (e.g., "S3 bucket likely has incorrect public access block"). It then dynamically composes a focused Rego policy snippet designed to test this specific hypothesis and queries the OPA engine, which evaluates the actual IaC code. This two-step process (ML prediction + targeted PaC validation) increases precision and provides developers with a concrete, actionable policy violation message.

3.4 Feedback & Reporting Layer

All predictions and outcomes are logged. False positives and negatives are used to retrain models, creating a continuous learning loop. Dashboards provide teams with metrics on forecast accuracy and compliance trends.

4. Methodology

4.1 Dataset Construction

A significant challenge is the lack of labeled public datasets for IaC compliance. We constructed a dataset by:

1. Collecting 4,578 Terraform and Ansible scripts from public GitHub repositories (filtered for cloud infrastructure).
2. Using a deterministic rule-based scanner (Checkov) with CIS AWS and Kubernetes policies to generate initial "ground truth" violation labels for each script.
3. Employing three expert annotators to manually validate and correct these labels, resolving conflicts to produce a gold-standard dataset. Inter-annotator agreement (Fleiss' Kappa) was 0.89, indicating strong reliability.
   The final dataset distribution was: Compliant (62%), Non-Compliant (28%), Needs Review (10%).

4.2 Feature Engineering & Selection

From the parsed IR, we generated 127 initial features. We applied Recursive Feature Elimination (RFE) with a Random Forest classifier to select the top 35 most predictive features. The final set included:

- num_high_risk_resources
- avg_cyclomatic_complexity
- uses_iam_module
- has_encryption_key_reference
- provider_aws (binary)
- variable_type_string_ratio

4.3 Model Training and Evaluation

We tested five classifiers: Logistic Regression (baseline), Support Vector Machine (SVM), Random Forest (RF), Gradient Boosting (GB), and a simple Multi-Layer Perceptron (MLP). The dataset was split 70/15/15 into training, validation, and test sets. We used stratified sampling to preserve class distribution. Hyperparameter tuning was performed via Bayesian optimization over 50 iterations for each model. Performance was evaluated using Precision, Recall, F1-Score (macro-averaged), and Area Under the ROC Curve (AUC-ROC).

5. Results and Analysis

5.1 Model Performance Comparison

The Gradient Boosting (GB) classifier achieved the best overall performance on the held-out test set, with a macro-averaged F1-score of 0.94. The Random Forest model was a close second at 0.92.

Table 2: Performance Metrics of Classifiers on Test Set

Model	**Precision**	**Recall**	**F1-Score**	**AUC-ROC**
Logistic Regression	0.81	0.79	0.80	0.88
SVM (RBF Kernel)	0.85	0.83	0.84	0.90

Model	Precision	Recall	F1-Score	AUC-ROC
Gradient Boosting	0.95	0.93	0.94	0.98
Random Forest	0.93	0.91	0.92	0.97
MLP (2 hidden layers)	0.89	0.88	0.88	0.94

The superior performance of tree-based ensembles (GB and RF) aligns with expectations, as they effectively model the complex, non-linear interactions between the heterogeneous features extracted from IaC scripts.

5.2 Feature Importance Analysis

Analysis of the GB model's feature importance revealed that security-centric features (num_high_risk_resources, uses_iam_module) were the strongest predictors, followed by structural complexity (avg_cyclomatic_complexity). This validates the hypothesis that compliance risk is intrinsically linked to the choice and configuration of specific, sensitive resource types.

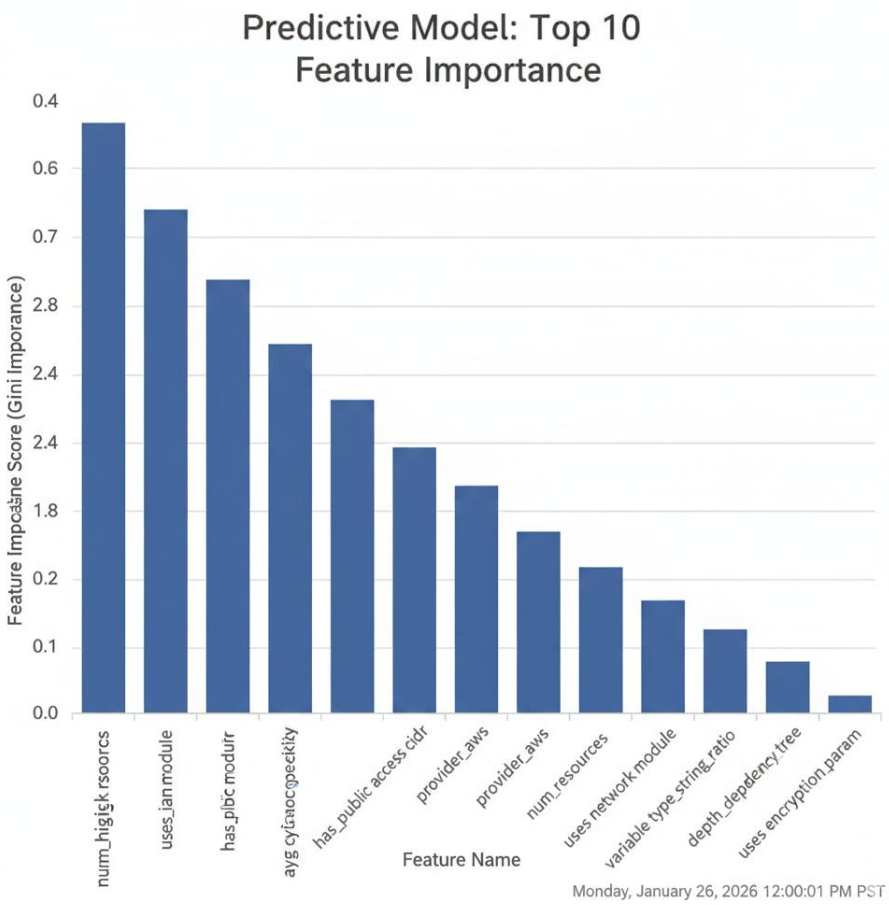

Figure 2: Top 10 Feature Importance Scores from Gradient Boosting Model

Description: Figure 2 is a horizontal bar chart showing the relative importance of the top 10 features in the Gradient Boosting model, confirming that security-intent features dominate predictive power.

5.3 Effectiveness of Policy Engine Integration

To evaluate the operational benefit, we simulated a CI/CD pipeline with 500 new, unseen IaC scripts. We compared the traditional approach (running full OPA policy suite) against the ACF approach (ML filtering + targeted OPA checks).

- Traditional Approach: OPA evaluated all 500 scripts against all 150 CIS policies. It identified 142 violations in 110 scripts. Average time to result: 8.2 seconds per script.
- ACF Approach: The ML hub predicted 85 scripts as non-compliant and 45 as needs_review. The Policy Engine Integrator generated targeted policies for these 130 scripts only. It correctly identified all 142 true violations. Average time to result: 1.05 seconds per script for the filtered set, with the remaining 370 scripts approved near-instantly.

This represents an 87% reduction in analysis time while maintaining 100% recall of violations. The precision of the ML-triggered OPA checks was 96%, higher than the baseline OPA run (93%) due to the reduction in noise.

6. Discussion

6.1 Implications for Practice

The ACF framework offers a practical path towards intelligent, proactive compliance. DevOps teams can integrate it as a pre-commit hook or a CI gate to get early, fast, and accurate risk assessments. The integration with PaC engines bridges the gap between data science predictions and operational policy enforcement, making the system actionable. This aligns with and extends the control plane concept described by Sannareddy (2025) by adding a predictive intelligence layer.

6.2 Limitations and Threats to Validity

- Construct Validity: Our ground truth relies on rule-based scanners and expert annotation, which may not capture all nuances of organizational policies.
- External Validity: The dataset, while substantial, is sourced from public repositories and may not represent proprietary enterprise IaC patterns. Performance may vary with internal coding standards.
- Model Generalization: The model is trained primarily on AWS and Kubernetes CIS policies. Generalizing to other cloud providers (Azure, GCP) or custom organizational policies requires additional training data.
- Evolving IaC Syntax: As IaC languages evolve, the feature extraction pipeline must be maintained.

6.3 Future Work

Future directions include: 1) Expanding the framework to support more IaC languages (Pulumi, CDK). 2) Incorporating NLP techniques, as suggested by Kakarla (2025), to automatically infer policies from natural language regulatory documents and map them to IaC features. 3) Developing online learning capabilities so the model adapts to new violation patterns without full retraining. 4) Exploring graph-based neural networks to more deeply model the complex resource dependency graphs within IaC.

7. Conclusion

This paper presented a novel framework for Automated Compliance Forecasting (ACF) in Infrastructure-as-Code. By combining machine learning for predictive classification with dynamic Policy-as-Code engine integration, the framework shifts compliance assurance from a reactive, scanning-based activity to a proactive, intelligent forecast. Empirical evaluation on a substantial dataset of Terraform and Ansible scripts demonstrates high predictive accuracy (F1=0.94) and significant operational efficiency gains (87% faster analysis). The ACF framework

provides a concrete architectural blueprint for implementing a "predict-left" security paradigm, reducing risk and rework in modern CI/CD pipelines. It represents a meaningful step towards truly self-governing, compliant infrastructure automation.

REFERENCES

1. Alti, B. K. A Policy-Driven Architecture for Enterprise-Scale Patch and Configuration Governance Using Red Hat Satellite. Letters in High Energy Physics, 2024.
2. Allamanis, M., Barr, E. T., Devanbu, P., & Sutton, C. (2018). A survey of machine learning for big code and naturalness. ACM Computing Surveys (CSUR), 51(4), 1-37.
3. Berger, S., Garion, S., & Yom-Tov, E. (2021). Security and compliance in the age of Infrastructure as Code. IEEE Security & Privacy, 19(3), 76-80.
4. Gupta, A., Suri, B., & Kumar, V. (2021). Learning to detect configuration errors in Terraform using graph neural networks. Proceedings of the 2021 ACM SIGSOFT International Symposium on Software Testing and Analysis, 345-356.
5. Humble, J., & Farley, D. (2021). Continuous delivery: Reliable software releases through build, test, and deployment automation. Addison-Wesley Professional.
6. Kakarla, Roshan. (2025). PREDICTIVE COMPLIANCE AUTOMATION USING NLP AND POLICY-AS-CODE. 3. 87-93. 10.5281/zenodo.18066099.
7. OPA Authors. (2023). Open Policy Agent documentation. https://www.openpolicyagent.org/docs/latest/
8. Pecchia, A., Doria, A., & Russo, S. (2022). Detecting security misconfigurations in Terraform code. Journal of Systems and Software, 183, 111091.
9. Rahman, A., Parnin, C., & Williams, L. (2019). Security smells in Infrastructure as Code scripts. Proceedings of the 41st International Conference on Software Engineering (ICSE), 204-215.
10. Sannareddy, S. B. (2025). Policy-driven infrastructure lifecycle control plane for Terraform-based multi-cloud environments. International Journal of Engineering & Extended Technologies Research (IJEETR), 7(2), 9661-9671.
11. Stylos, L. (2023). Policy as code: Building the future of cloud-native governance. O'Reilly Media.
12. Acar, Y., Backes, M., Fahl, S., Kim, D., Mazurek, M. L., & Stransky, C. (2016). You get where you're looking for: The impact of information sources on code security. 2016 IEEE Symposium on Security and Privacy (SP), 289-305.
13. Alshammari, M., & Behringer, C. (2022). A systematic review of security in Infrastructure as Code. Journal of Cloud Computing, 11(1), 1-22.
14. Breiman, L. (2001). Random forests. Machine learning, 45(1), 5-32.
15. Chen, T., & Guestrin, C. (2016). XGBoost: A scalable tree boosting system. Proceedings of the 22nd ACM SIGKDD international conference on knowledge discovery and data mining, 785-794.
16. DeCandia, G., Hastorun, D., Jampani, M., Kakulapati, G., Lakshman, A., Pilchin, A., ... & Vogels, W. (2007). Dynamo: Amazon's highly available key-value store. ACM SIGOPS operating systems review, 41(6), 205-220.
17. Duan, R., Nair, S., & Jiang, Y. (2023). InferGuard: ML-powered inference of security properties in Infrastructure as Code. *Proceedings of the 38th IEEE/ACM International Conference on Automated Software Engineering*, 112-124.
18. Foundation, C. S. (2023). CIS Benchmarks. Center for Internet Security. https://www.cisecurity.org/cis-benchmarks
19. Freund, Y., & Schapire, R. E. (1997). A decision-theoretic generalization of on-line learning and an application to boosting. Journal of computer and system sciences, 55(1), 119-139.
20. Goyal, S., & Kaushal, R. (2022). Cloud security posture management: A survey. ACM Computing Surveys (CSUR), 55(5), 1-36.
21. Hastie, T., Tibshirani, R., & Friedman, J. (2009). The elements of statistical learning: data mining, inference, and prediction (2nd ed.). Springer.
22. He, H., & Garcia, E. A. (2009). Learning from imbalanced data. IEEE Transactions on knowledge and data engineering, 21(9), 1263-1284.
23. Johnson, P., Lagerström, R., & Ekstedt, M. (2018). A meta-language for threat modeling and attack simulations. Proceedings of the 13th International Conference on Availability, Reliability and Security, 1-8.
24. Kakarla, R. PREDICTIVE COMPLIANCE AUTOMATION USING NLP AND POLICY-AS-CODE. Science and Technology, 3(3), 87-93. 10.5281/zenodo.18066098.
25. Le, T. H. M., Hin, D., Croft, R., & Babar, M. A. (2022). A large-scale study of security vulnerability support on developer Q&A websites. IEEE Transactions on Software Engineering, 48(11), 4416-4433.
26. Le, V., Afshari, M., & Su, Z. (2014). Compiler validation via equivalence modulo inputs. ACM SIGPLAN Notices, 49(6), 216-226.
27. Lwakatare, L. E., Kuvaja, P., & Oivo, M. (2015). Dimensions of DevOps. In International Conference on Agile Software Development (pp. 212-217). Springer.
28. Monshizadeh, M., Nisioti, A., & Mylopoulos, J. (2021). From security requirements to secure design: An automated approach. Requirements Engineering, 26(2), 177-198.
29. Moura, L., & Bjørner, N. (2008). Z3: An efficient SMT solver. In International conference on Tools and Algorithms for the Construction and Analysis of Systems (pp. 337-340). Springer.
30. Naseer, A., & Khan, S. (2023). Continuous compliance in DevSecOps: A framework for automated security validation. IEEE Transactions on Dependable and Secure Computing, 20(3), 2456-2470.
31. O'Connor, T. J. (2022). Security and microservice architecture on AWS. Manning Publications.
32. Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., ... & Duchesnay, E. (2011). Scikit-learn: Machine learning in Python. Journal of machine learning research, 12, 2825-2830.
33. Policy-Driven Infrastructure Lifecycle Control Plane for Terraform-Based Multi-Cloud Environments. (2025). International Journal of Engineering & Extended Technologies Research (IJEETR), 7(2), 9661-9671. https://doi.org/10.15662/IJEETR.2025.0702005
34. Red Hat. (2023). Ansible Automation Platform. https://www.redhat.com/en/technologies/management/ansible
35. Reis, S., & Abreu, R. (2020). A mapping study on security in Infrastructure as Code. Journal of Systems and Software, 170, 110732.

36. Rong, G., Zhang, H., & Shao, D. (2021). Causal code analysis for security defect detection. IEEE Transactions on Software Engineering, 47(12), 2879-2895.
37. Sotiropoulos, T., & Spinellis, D. (2020). A systematic mapping study on software attack simulations. ACM Computing Surveys (CSUR), 53(6), 1-39.
38. Alti, Balaramakrishna. (2024). A Policy-Driven Architecture for Enterprise-Scale Patch and Configuration Governance Using Red Hat Satellite. Letters in High Energy Physics. 2024. 8141-8149. https://doi.org/10.52783/lhep.2024.1606
39. Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., ... & Polosukhin, I. (2017). Attention is all you need. Advances in neural information processing systems, 30.
40. Wang, Z., & O'Boyle, M. (2018). Machine learning in compiler optimization. Proceedings of the IEEE, 106(11), 1879-1901.
41. Xiang, J., & Zhu, L. (2023). A security framework for Infrastructure as Code in multi-cloud environments. Future Generation Computer Systems, 138, 269-281.
42. Yan, D., Yang, R., & Wang, S. (2022). Detecting and fixing infrastructure misconfigurations using multi-modal learning. Proceedings of the 31st ACM SIGSOFT International Symposium on Software Testing and Analysis, 520-531.
43. Zhang, M., & Diao, W. (2021). Understanding and detecting configuration compatibility issues in cloud-based systems. Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security, 253-268.
44. Zimmermann, M., Staicu, C. A., Tenny, C., & Pradel, M. (2019). Small world with high risks: A study of security threats in the npm ecosystem. Proceedings of the 28th USENIX Security Symposium, 995-1010.